

计算机系统2期末复习-计科01

1.信息的表示和处理（第二章）

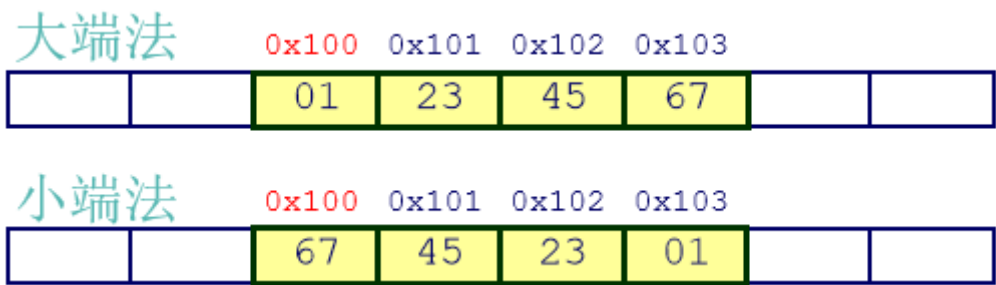
1.1信息存储

- 进制转换：
 - 十六进制、十进制、八进制、二进制互转
- 数据的大小：字（word）是计算机中固定长度的二进制位，用于指明整数和指针数据的标称大小。常见的字长有32位和64位。C语言中不同数据类型在32位和64位机器上的大小有所不同。

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
long double	–	–	10/16
pointer	4	8	8

- 寻址和字节顺序：

字节顺序：大端法（最高有效字节在前面，如Sun, PPC Mac, Internet）和小端法（最低有效字节在前面，如x86）。双端法（ARM）可以将处理器配置成小端或大端两种方式。变量x的值为0x01234567，存储的起始地址为0x100，不同的机器类型的字节顺序如下所示：

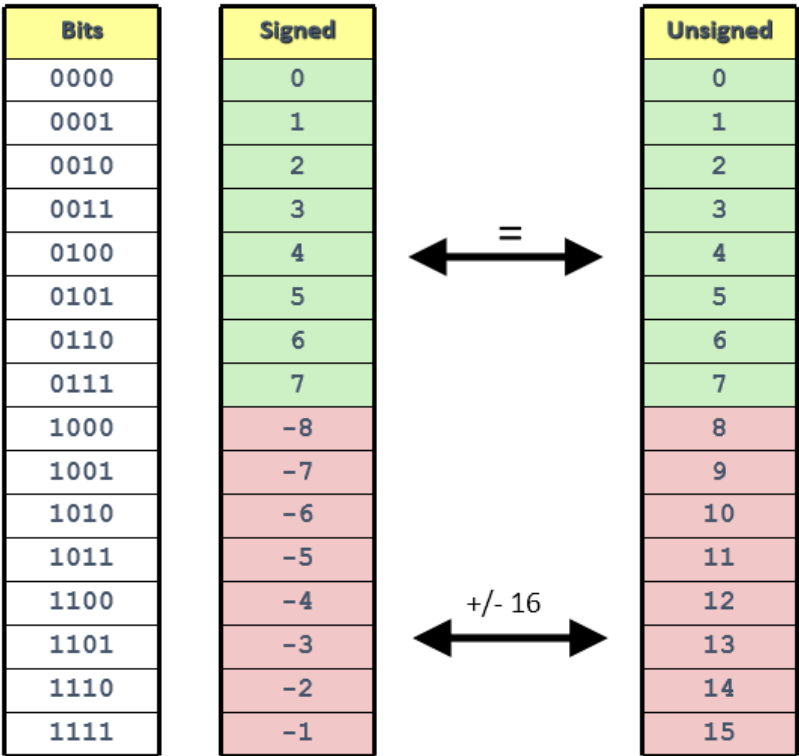


- **布尔代数简介**：基于二元集合{0, 1}或{TRUE, FALSE}的代数，包含逻辑运算AND、NOT、XOR、OR等。
- **C语言中的位级运算**：包括NOT (~)、AND (&)、OR (|)、XOR (^) 等位级运算符，用于对二进制位进行操作。
- **C语言中的逻辑运算**：提供逻辑运算符||、&&和!，分别对应于OR、AND和NOT运算。逻辑运算中，所有非零的参数都表示TRUE，参数零表示FALSE。

- **C语言中的移位运算**：包括左移 (<<) 和右移 (>>)。逻辑右移在左端补0，算术右移在左端补最高有效位的值。

1.2整数表示

- **无符号数的编码**：编码范围为0到 2^w-1 。
- **补码编码**：编码范围为 $-2^{(w-1)}$ 到 $2^{(w-1)}-1$ 。
- **补码与模运算**：在一个模运算系统中，一个数与它除以“模”后的余数等价。补码可以看作是模 2^w 的系统中的表示。
- **有符号数和无符号数之间的转换**：C语言允许强制类型转换，相同字长的类型转换，数值可能改变，但不改变“位模式”。



- **C语言中的有符号数与无符号数**：常数默认是有符号的，创建无符号常量需要加上后缀字符‘U’。表达式中既有无符号数又有有符号数时，**有符号数会隐式转换为无符号数**。

关系表达式	类型	结果	说明
0 == 0U	无	1	00...0B = 00...0B
-1 < 0	带	1	11...1B (-1) < 00...0B (0)
-1 < 0U	无	0*	11...1B (2 ³² -1) > 00...0B(0)
2147483647 > -2147483647 - 1	带	1	011...1B (2 ³¹ -1) > 100...0B (-2 ³¹)
2147483647U > -2147483647 - 1	无	0*	011...1B (2 ³¹ -1) < 100...0B(2 ³¹)
2147483647 > (int) 2147483648U	带	1*	011...1B (2 ³¹ -1) > 100...0B (-2 ³¹)
-1 > -2	带	1	11...1B (-1) > 11...10B (-2)
(unsigned) -1 > -2	无	1	11...1B (2 ³² -1) > 11...10B (2 ³² -2)

- **扩展一个数字的位表示**：无符号数的零扩展和有符号数的符号扩展。

1.3 整数运算

- 加法
 - **无符号加法**：标准的加法函数，忽略进位输出，实现模块化的算法。
 - 若溢出，则结果必定小于两个操作数
 - **补码加法**：有符号整数的加法，结果**可能溢出**。
 - 正数+正数=负数
 - 负数+负数=正数
 - **补码的非**：在C语言中，对于任意整数值x，计算表达式-x和~x+1得到的结果完全一样。
- 乘法
 - **无符号乘法**：两数（n位）相乘，结果（2n位）截低（n）位。
 - **补码乘法**：两个有符号整数相乘，结果为有符号整数。

$$x^2 \geq 0?$$

对于带符号整数，不一定！

例如，当n=4时， $5^2 = -7 < 0!$

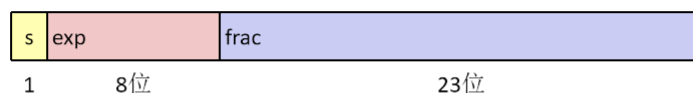
11001

- **乘以常数**：以移位、加法和减法的组合来消除整数乘以常数的情况。
- 除法
 - 无符号数的除法可以使用**逻辑右移**。
 - 有符号数的除法可以使用**算术右移**。

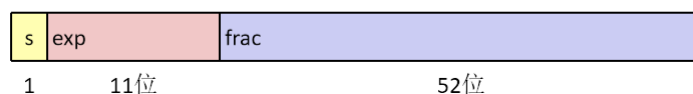
1.4 浮点数

- **科学计数法与浮点数**：浮点数的表示类似于科学计数法，包括符号位、尾数和指数。
- **二进制小数**：二进制小数的形式为b_mb_{m-1}...b₁b₀.b₋₁b₋₂...b_{-n}，其中b_i为0或1。
- **IEEE浮点表示**：IEEE754标准定义了浮点数的表示方式，包括单精度（32位）和双精度（64位）。

■ 单精度: 32 位

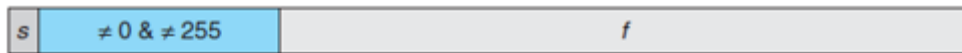


■ 双精度: 64 位

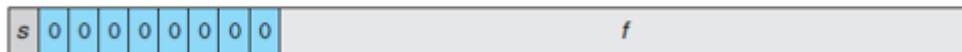


- **浮点表示范围**：浮点数的表示范围比定点数大，但数之间更稀疏，且不均匀。
- **规格化数与非规格化数**：
 - 规格化数：exp字段不全为0且不全为1，阶码的值为E=e-bias，尾数M=1+f。
 - 非规格化数：exp字段全为0，E=1-bias，M=f。
- **特殊值**：
 - 零：exp字段全为0，尾数字段全为0。
 - 无穷大：exp字段全为1，尾数字段全为0。
 - NaN（非数）：exp字段全为1，尾数字段不全为0。

1. Normalized



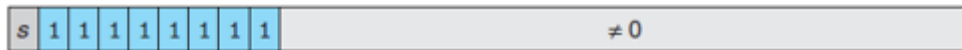
2. Denormalized



3a. Infinity



3b. NaN



- **数字示例：**通过具体的例子展示了浮点数的表示和转换。

1 011 11101 110 0000 0000 0000 0000 0000

$$(-1)^s \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-127)}$$

° **Sign:** 1 => negative

° **Exponent:**

- $0111\ 1101_{\text{two}} = 125_{\text{ten}}$

- Bias adjustment: $125 - 127 = -2$

° **Significand:**

$$1 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4} + 0 \times 2^{-5} + \dots$$

$$= 1 + 2^{-1} + 2^{-2} = 1 + 0.5 + 0.25 = 1.75$$

° **Represents:** $-1.75_{\text{ten}} \times 2^{-2} = -0.4375$

- **舍入：**由于浮点数位数有限，存在舍入问题。IEEE规定了四种舍入方式：向0舍入、向下舍入、向上舍入以及向偶数舍入。
- **浮点运算：**
 - 浮点加法：高阶对齐、尾数加减、规格化、舍入。

$$1.010 \times 2^2 + 1.110 \times 2^3 = (0.101 + 1.110) \times 2^3$$

$$= 10.011 \times 2^3 = 1.010 \times 2^4$$

- 浮点乘法：符号位异或、尾数相乘、阶码相加、规格化、舍入。

$$\text{保留4位有效数字: } 1.010 \times 2^2 \times 1.110 \times 2^3 = 10.0011 \times 2^5 =$$

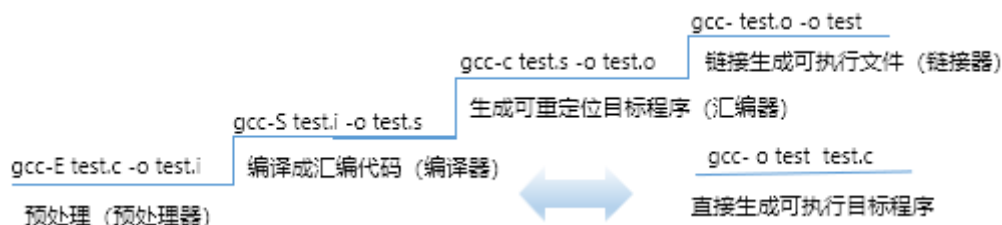
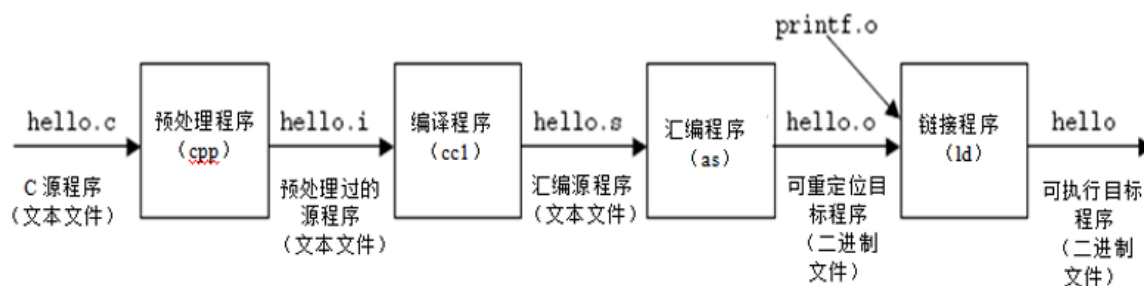
$$1.00011 \times 2^6 = 1.001 \times 2^6$$

- **C语言中的浮点数：**C语言中有float和double两种浮点数类型，C99还包含long double类型。整数和浮点数之间可以进行转换，但可能会有精度损失或溢出。

2.程序的机器级表示（第三章）

2.1程序转换概述

- 用GCC编译器套件进行转换的过程
 - 预处理 (Preprocessing)：在源程序中进行宏定义展开、条件编译指令处理和包含头文件，输出.i文件。
 - 编译 (Compilation)：将预处理后的源程序文件编译生成相应的汇编语言程序，输出.s文件。
 - 汇编 (Assembly)：由汇编程序将汇编语言源程序文件转换为可重定位的机器语言目标代码文件，即.o文件。 **(地址从0开始)**
 - 链接 (Linking)：由链接器将多个可重定位的机器语言目标文件以及库例程（如printf()库函数）链接起来，生成最终的可执行目标文件，通常为.exe (Win) 或无后缀 (Linux)。



2.2IA-32/x86-64指令系统

- x86-64整数寄存器

%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

- 操作数

C 声明	Intel 数据类型	汇编代码后缀	大小(字节)
char	字节	b	1
short	字	w	2
int	双字	l	4
long	四字	q	8
char*	四字	q	8
float	单精度	s	4
double	双精度	l	8

- 寻址方式

寻址方式	说明		
立即寻址	指令直接给出操作数		
寄存器寻址	指定的寄存器R的内容为操作数		
位移	$LA = (SR) + A$		存储器操作数
基址寻址	$LA = (SR) + (B)$		
基址加位移	$LA = (SR) + (B) + A$		
比例变址加位移	$LA = (SR) + (I) \times S + A$		
基址加变址加位移	$LA = (SR) + (B) + (I) + A$		
基址加比例变址加位移	$LA = (SR) + (B) + (I) \times S + A$		
相对寻址	$LA = (PC) + A$	跳转目标指令地址	

2.2.1 数据传送指令

- mov (movb/movw/movl/movq对应1/2/4/8字) 源, 目的

源类型	目的类型	Src, Dest	对应的C语句
movq {	立即数 { 寄	movq \$0x4, %rax	temp = 0x4;
	内 { 内	movq \$-147, (%rax)	*p = -147;
	寄存器 { 寄	movq %rbx, %rdx	temp2 = temp1;
		movq %rax, (%rdx)	*p = temp;
	内存 { 寄	movq (%rax), %rdx	temp = *p;

- 无法使用单个指令从内存到内存的数据传输

2.2.2 算术运算指令

- 两元操作 (注意src和dest的顺序)

	格式	计算	
addq	Src, Dest	Dest = Dest + Src	
subq	Src, Dest	Dest = Dest - Src	
imulq	Src , Dest	Dest = Dest * Src	
salq	Src, Dest	Dest = Dest << Src	(shlq)
sarq	Src, Dest	Dest = Dest >> Src	(算术)
shrq	Src, Dest	Dest = Dest >> Src	(逻辑)
xorq	Src, Dest	Dest = Dest ^ Src	
andq	Src, Dest	Dest = Dest & Src	
orq	Src, Dest	Dest = Dest Src	

- 一元操作

一元操作

incq	Dest	Dest = Dest + 1
decq	Dest	Dest = Dest - 1
negq	Dest	Dest = - Dest
notq	Dest	Dest = ~Dest

2.2.3 移位操作

- SAL/SHL/SAR/SHR (区分逻辑移位和算术移位)
- 移位量是立即数或者%cl

1 movq %rdi, %rax	Get x
2 salq \$4, %rax;	x <<= 4
3 movl %esi, %ecx	Get n
4 sarq %cl, %rax;	x >>= n

2.2.4 控制

- 条件码
 - CF: Carry Flag. 进位标志 (无符号数的溢出)
 - ZF: Zero Flag. 结果为零标志
 - SF: Sign Flag. 符号标志 (结果为负时标志位为1)
 - OF: Overflow Flag. 溢出标志 (正溢出或负溢出)
 - leaq不改变条件码
- 访问条件码
 - 基于条件码的组合将目标的低位字节设置为0或1
 - 不改变剩余7字节

SetX	效果	设置条件
sete	ZF	相等/零
setne	~ZF	不等/非零
sets	SF	负数
setns	~SF	非负数
setg	$\sim(SF \wedge OF) \ \& \sim ZF$	大于(有符号>)
setge	$\sim(SF \wedge OF)$	大于等于(有符号>=)
setl	$(SF \wedge OF)$	小于(有符号<)
setle	$(SF \wedge OF) \mid ZF$	小于等于(有符号<=)
seta	$\sim CF \ \& \sim ZF$	超过(无符号>)
setae	~CF	超过或相等(无符号>=)
setb	CF	低于(无符号<)
setbe	CF $\mid$ ZF	低于或相等(无符号<=)

- 简单示例：通常使用movzbl完成工作

<pre>int gt (long x, long y) { return x >= y; ? }</pre>	寄存器	用于
	%rdi	x
	%rsi	y
	%eax	返回结果值

cmpq	%rsi, %rdi	# Compare x:y
setge	%al	# Set when >
movzbl	%al, %eax	# Zero rest of %rax
ret		

- 条件跳转指令
 - 根据条件码跳转到代码不同部分

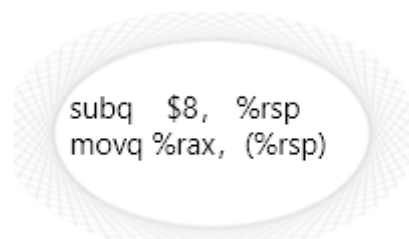
指令	同义名	跳转条件	描述
jmp <i>Label</i>		1	直接跳转
jmp <i>*Operand</i>		1	间接跳转
j _e <i>Label</i>	j _z	ZF	相等/零
j _{ne} <i>Label</i>	j _{nz}	~ZF	不相等/非零
j _s <i>Label</i>		SF	负数
j _{ns} <i>Label</i>		~SF	非负数
j _g <i>Label</i>	j _{nle}	$\sim(SF \wedge OF) \ \& \sim ZF$	大于(有符号>)
j _{ge} <i>Label</i>	j _{nl}	$\sim(SF \wedge OF)$	大于或等于(有符号>=)
j _l <i>Label</i>	j _{nge}	$SF \wedge OF$	小于(有符号<)
j _{le} <i>Label</i>	j _{ng}	$(SF \wedge OF) \mid ZF$	小于或等于(有符号<=)
j _a <i>Label</i>	j _{nbe}	$\sim CF \ \& \sim ZF$	超过(无符号>)
j _{ae} <i>Label</i>	j _{nb}	~CF	超过或相等(无符号>=)
j _b <i>Label</i>	j _{nae}	CF	低于(无符号<)
j _{be} <i>Label</i>	j _{na}	CF $\mid$ ZF	低于或相等(无符号<=)

- 条件传送指令
 - 与条件跳转不同，处理器无需预测测试结果就可以执行条件传送。
 - 处理器只是读源值（可能是从内存中），检查条件码，然后要么更新目的寄存器，要么保持不变。

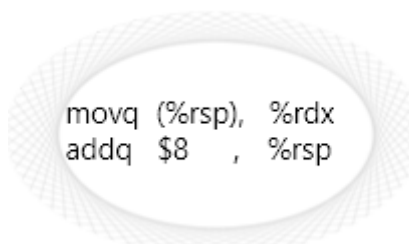
指令	同义名	传送条件	描述
<code>cmovz S, R</code>	<code>cmovz</code>	ZF	相等/零
<code>cmovne S, R</code>	<code>cmovnz</code>	$\sim$ ZF	不相等/非零
<code>cmovs S, R</code>		SF	负数
<code>cmovns S, R</code>		$\sim$ SF	非负数
<code>cmovg S, R</code>	<code>cmovnle</code>	$\sim$ (SF $\wedge$ OF) & $\sim$ ZF	大于 (有符号>)
<code>cmovge S, R</code>	<code>cmovnl</code>	$\sim$ (SF $\wedge$ OF)	大于或等于 (有符号>=)
<code>cmovl S, R</code>	<code>cmovnge</code>	SF $\wedge$ OF	小于 (有符号<)
<code>cmovle S, R</code>	<code>cmovng</code>	(SF $\wedge$ OF) ZF	小于或等于 (有符号<=)
<code>cmova S, R</code>	<code>cmovnbe</code>	$\sim$ CF & $\sim$ ZF	超过 (无符号>)
<code>cmovae S, R</code>	<code>cmovnb</code>	$\sim$ CF	超过或相等 (无符号>=)
<code>cmovb S, R</code>	<code>cmovnae</code>	CF	低于 (无符号<)
<code>cmovbe S, R</code>	<code>cmovna</code>	CF ZF	低于或相等 (无符号<=)

2.2.5栈

- 先进后出，从高地址向低地址增长
- 入栈 (`pushq %rax`)

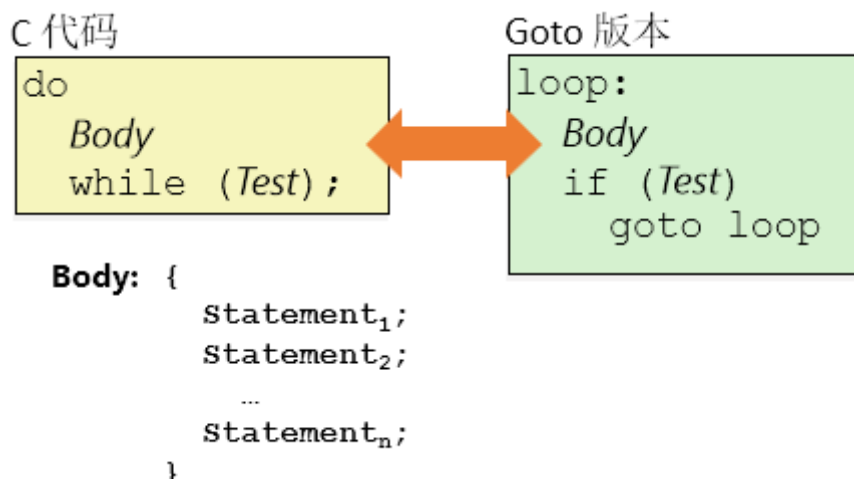


- 出栈 (`popq %rdx`)



2.2.6循环

- 一般的“Do-While”翻译方法



- 一般的“While”翻译方法

- “Jump-to-middle”
- 与 -Og 一起使用

While 版本

```
while (Test)
    Body
```



Goto 版本

```
goto test_Label;
loop:
    Body
test_Label:
    if (Test)
        goto loop;
done:
```

- for循环

for (初始化; 判断; 更新)
代码主体

```
#define WSIZE 8*sizeof(long)
long pcount_for
(unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < WSIZE; i++)
    {
        unsigned bit =
            (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

初始化

```
i = 0
```

判断

```
i < WSIZE
```

更新

```
i++
```

代码主体

```
{
    unsigned bit =
        (x >> i) & 0x1;
    result += bit;
}
```

2.2.7跳转

- 跳转表

Switch 形式

```
switch(x) {
    case val_0:
        Block 0
    case val_1:
        Block 1
        . . .
    case val_n-1:
        Block n-1
}
```

跳转表

JTab: *Targ0*
Targ1
Targ2
.
.
.
Targn-1

跳转目标块

Targ0: Code Block 0

Targ1: Code Block 1

Targ2: Code Block 2

⋮

Targn-1: Code Block n-1

翻译 (C扩展)

```
goto *JTab[x];
```

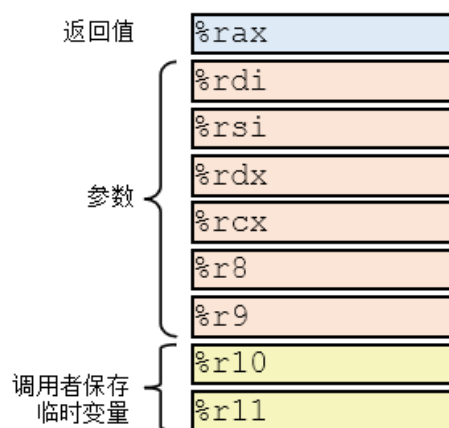
2.3C语言程序的机器级表示

2.3.1过程调用机制

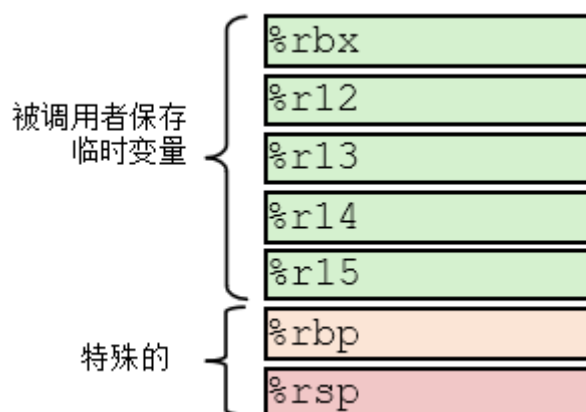
- 传递控制
 - 进入过程Q，PC设置为Q的起始地址，栈中保存返回地址（call target）
 - 返回时，PC设置为P中调用Q后面那条指令地址（ret）
- 传递数据
 - P向Q提供一个或多个数据
 - Q必须能够向P返回一个值
- 内存管理
 - Q为局部变量分配空间
 - 返回前释放存储空间

2.3.2寄存器用法

- 以下寄存器均可以用于调用者储存，可以被过程修改
- %rax可用于返回值



- 用于被调用者保存，被调用者必须保存和回复
- %rbp可能作为帧指针使用



- 其他寄存器

%rax	返回值	%r8	参数 #5
%rbx	被调用者保存	%r9	参数 #6
%rcx	参数 #4	%r10	调用者保存
%rdx	参数 #3	%r11	调用者保存
%rsi	参数 #2	%r12	被调用者保存
%rdi	参数 #1	%r13	被调用者保存
%rsp	栈指针	%r14	被调用者保存
%rbp	被调用者保存	%r15	被调用者保存

2.4复杂数据类型的分配和访问

- 数组的分配和访问

数组定义	数组名	数组元素类型	数组元素大小 (B)	数组大小 (B)	起始地址	元素 i 的地址
char S[10]	S	char	1	10	&S[0]	&S[0]+i
char * SA[10]	SA	char *	4	40	&SA[0]	&SA[0]+4*i
double D[10]	D	double	8	80	&D[0]	&D[0]+8*i
double * DA[10]	DA	double *	4	40	&DA[0]	&DA[0]+4*i

- 结构的对齐和访问

对齐

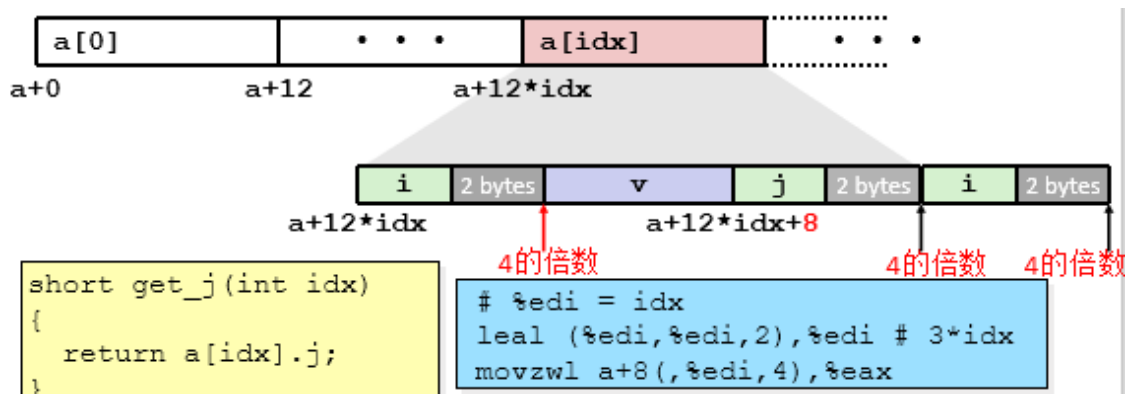
- 结构数组变量的最末可能需要插空，以使每个数组元素都按4字节边界对齐
- 最简单的对齐策略是，按其数据长度进行对齐，例如，int型地址是4的倍数，short型地址是2的倍数，double和long long型的是8的倍数，float型的是4的倍数，char不对齐。Windows 采用该策略。
- Linux策略更宽松：short是2的倍数，其他如int、double和指针等都是4的倍数。

```
struct SDT {
    int    i;
    short  si;
    double d;
    char   c;
} sa[10];
```



指针表示结构的第一个字节

访问带偏移的元素



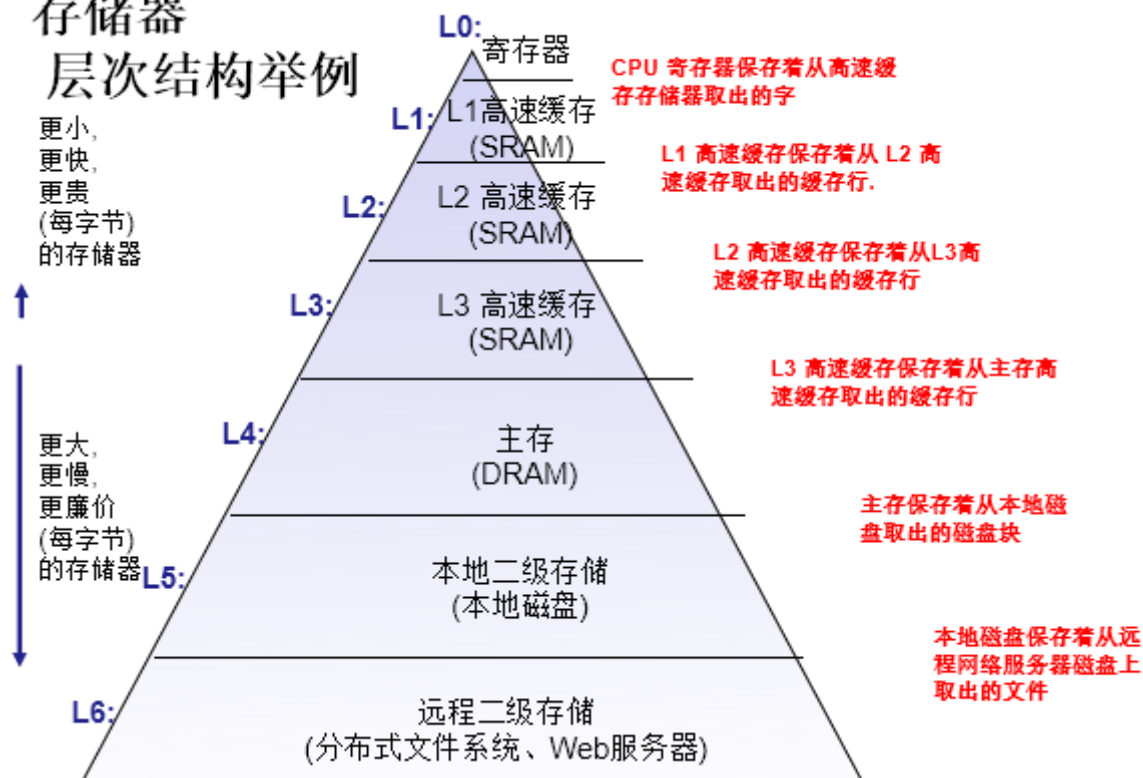
2.5越界访问和缓冲区溢出

- 缓冲区溢出
 - 解释：超越数组存储区范围的写入操作称为缓冲区溢出
 - 常见形式：字符串输入时未检查长度
- 防止溢出漏洞
 - 限制字符串长度的库例程
 - fgets 代替 gets
 - strncpy 代替 strcpy
 - 不要用 scanf %s 转换规格，使用fgets读取字符串，或者使用% ns，其中n是一个合适的整数。
 - 采用系统级保护
 - 在程序开始时，在栈上分配任意数量的空间，移位整个程序的堆栈地址。
 - 标记存储区域为只读或者可写
 - 栈金丝雀
 - 在栈帧中**任何局部缓冲区**与栈状态间存储一个特殊的金丝雀值（哨兵值）
 - 从函数返回前，程序检查这个金丝雀值是否改变了

3.储存器层次结构（第六章）

3.1储存技术及趋势

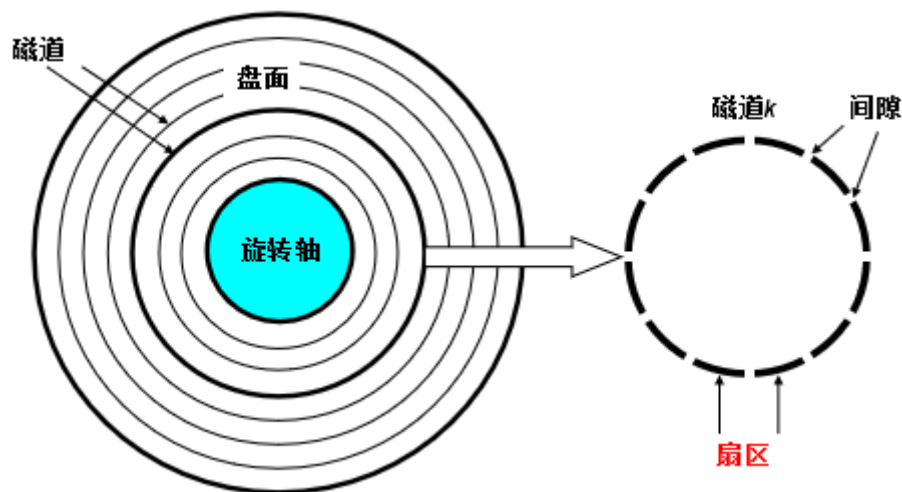
存储器 层次结构举例



- 随机访问存储器
 - SRAM与DSRAM
 - SRAM: 静态RAM- Static RAM, SRAM使用触发器 (通常由6个晶体管组成) 来存储每个比特的数据。这种设计使得存储单元在没有外部刷新操作的情况下能够保持数据状态, 因此称为“静态”存储器。
 - DRAM: 动态RAM— Dynamic RAM, DRAM使用一个晶体管和一个电容器来存储每个比特的数据。电容器会随着时间的逐渐漏电, 因此需要定期刷新 (通常每几毫秒刷新一次) 以保持数据状态, 因此称为“动态”存储器。
 - RAM封装在芯片上, 每个单元储存1bit

芯片类型	晶体管数	相对访问时间	持续的	敏感的	相对花费	应用
SRAM	4 或 6个	1X	是	否	1000X	高速缓存存储器
DRAM	1个	10X	否	是	1X	主存, 帧缓冲区

- 非易失性存储器
 - 只读存储器(ROM): 生产时写入程序, 只能写一次
 - Programmable ROM (PROM): 可以重新一次编程
 - Erasable PROM (EPROM): 可用紫外线或X光整块擦除
 - Electrically erasable PROM (EEPROM): 可用电子整块擦除
 - Flash memory: EEPROMs, 以块为单位进行擦除, 100000次擦除后即磨损坏
- 磁盘
 - 结构
 - 磁盘由双面的盘片组成
 - 每张盘面上密集地排布着环形磁道
 - 每条磁道上有多扇区, 每个扇区由间隙隔开

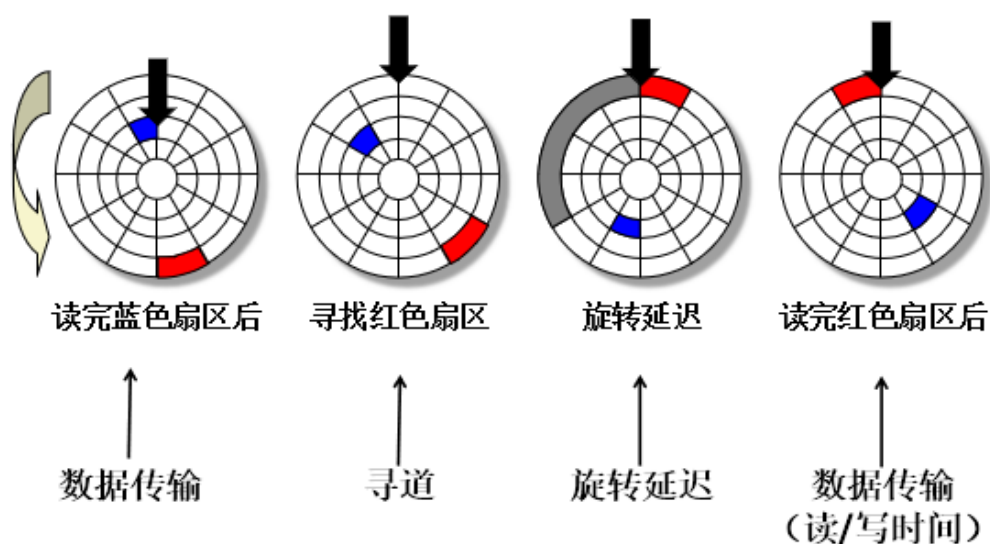


容量

- 计算: (字节数/扇区) × (平均扇区数/磁道) × (磁道数/盘面) × (盘面数/盘片) × (盘片/磁盘)
- 举例:
 - 512 字节/扇区
 - 300 扇区/磁道 (平均值)
 - 20,000 磁道/盘面
 - 2 盘面/盘片
 - 5 盘片/磁盘
- 计算容量:
 - = $512 \times 300 \times 20000 \times 2 \times 5$
 - = 30,720,000,000
 - = 30.72 GB

访问时间

- 时间的组成: 寻道时间+旋转时间+数据传输时间



寻道时间

- 磁头由一个柱面移动到另一个柱面的时间
- 通常寻道时间为: 3-9ms

旋转时间

- 经过磁盘旋转, 目标扇区到达磁头下的时间
- 最大旋转延迟 = $1/\text{RPMs} \times 60 \text{ sec}/1 \text{ min}$
- 平均旋转延迟 = $0.5 \times \text{最大旋转延迟}$
- 通常旋转速率 = 7200 RPMs

- 数据传输时间
 - 传输每个扇区所需时间
 - 数据传输时间 = $1/\text{RPM} \times 1/(\text{平均扇区数/磁道}) \times 60 \text{ 秒}/1 \text{ 分钟}$
- 举例：

旋转速度 = 7,200 RPM
 平均寻道时间 = 9ms
 平均扇区数/磁道 = 400
- 计算结果：

平均旋转时间 = $1/2 \times (60 \text{ secs}/7200 \text{ RPM}) \times 1000\text{ms/sec} \approx 4\text{ms}$.
 数据传输时间 = $60/7200 \text{ RPM} \times 1/400 \text{ secs/track} \times 1000\text{ms/sec} \approx 0.02\text{ms}$
 服务总时间 = $9\text{ms} + 4\text{ms} + 0.02\text{ms} = 13.02\text{ms}$

3.2局部性

- 待解决问题：在DRAM、disk、和CPU间，速度差距大
- 局部性原理
 - 程序倾向于使用最近一段时间，距离其较近地址的数据和指令
- 局部性例子
 - 数据引用
 - 空间：连续引用数组元素（步长为1的引用模式）
 - 时间：每次迭代引用的变量sum
 - 指令引用
 - 空间：顺序引用指令
 - 时间：重复地使用loop循环

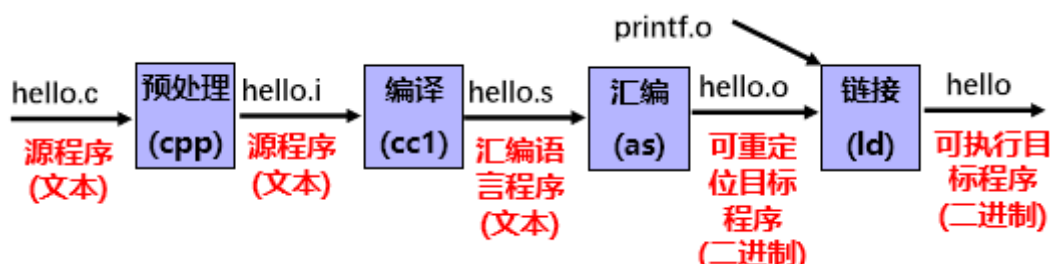
3.3储存器层次结构中的高速缓存

- Cache简介
 - 用途：一种更小，速度更快的存储设备。作为更大、慢存储设备的缓冲区。 **解决CPU与主存之间速度匹配的问题**
 - 实现：储存容量小，以**关联存储器**方式运行、用**静态存储器芯片**实现的存储器系统
 - 要求：有足够高的命中率
- Cache映射
 - 直接相联(Direct)：每个主存块映射到Cache的固定行
 - 全相联(Full Associate)：每个主存块映射到Cache的任一行
 - 组相联(Set Associate)：每个主存块映射到Cache固定组中任一行，最常用

4.链接（第七章）

4.1目标格式文件

- 经典程序转换处理过程

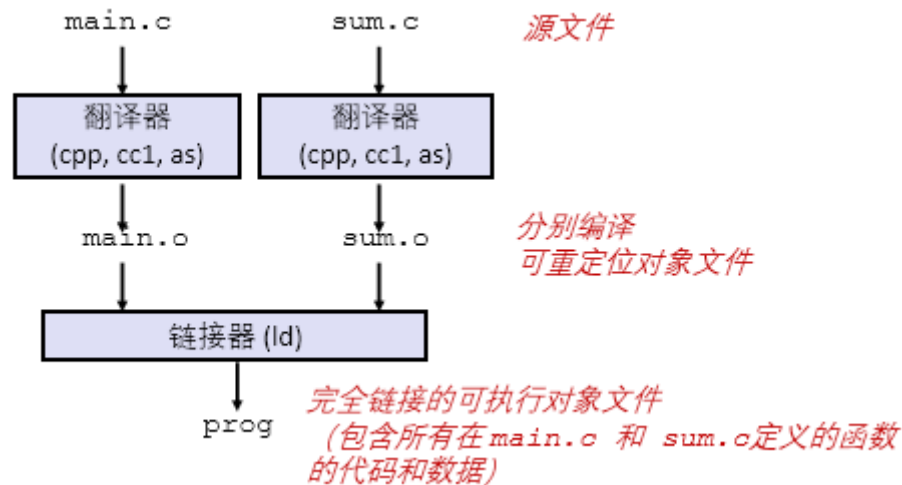


- 静态链接

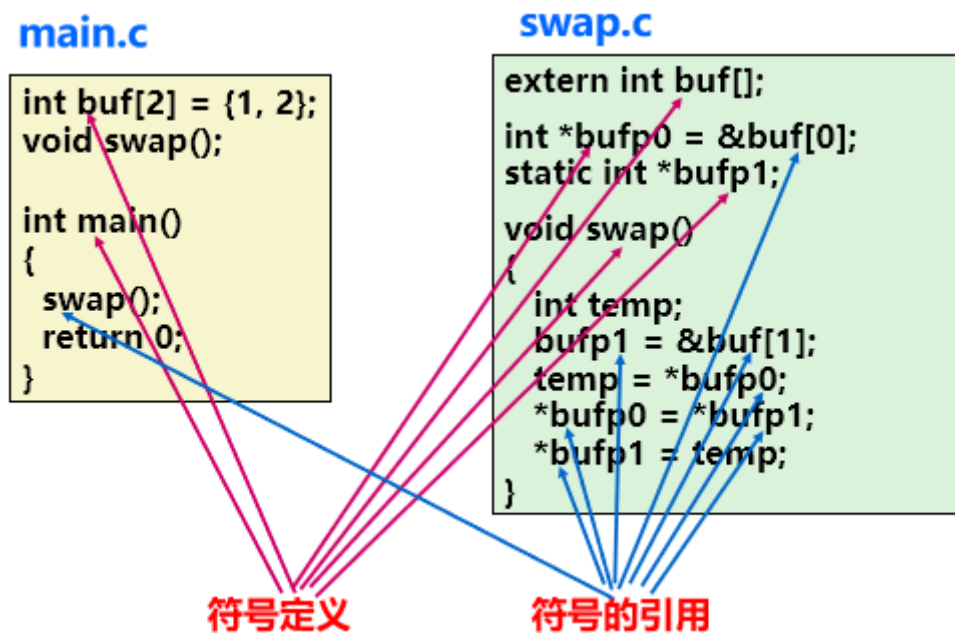
- 使用编译器驱动程序翻译和链接程序:

linux> gcc -Og -o prog main.c sum.c

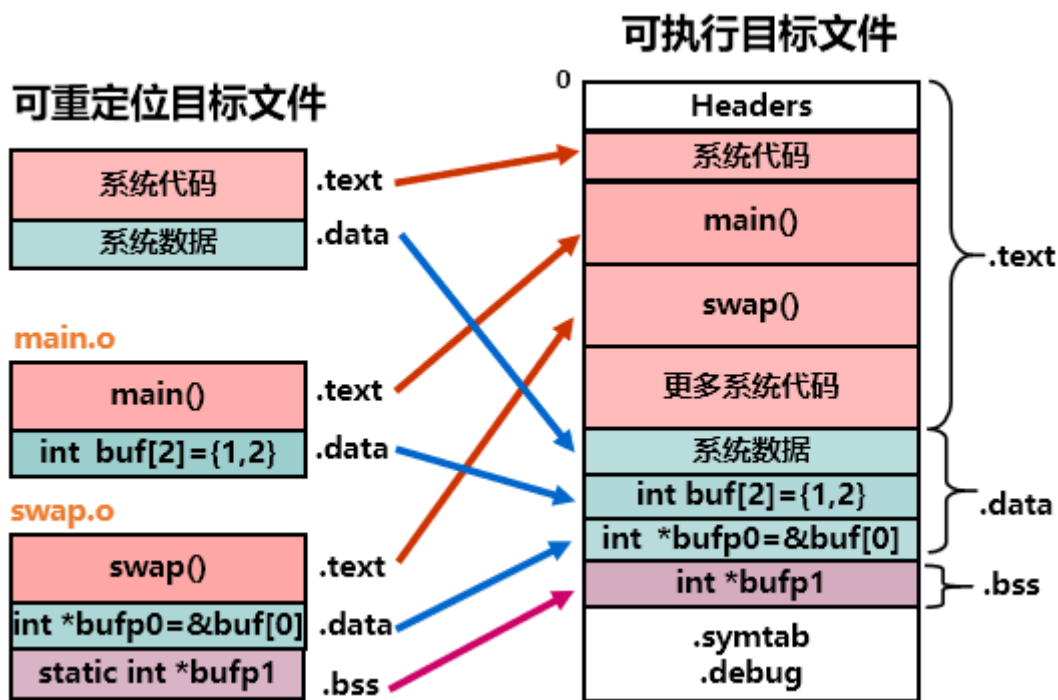
linux> ./prog



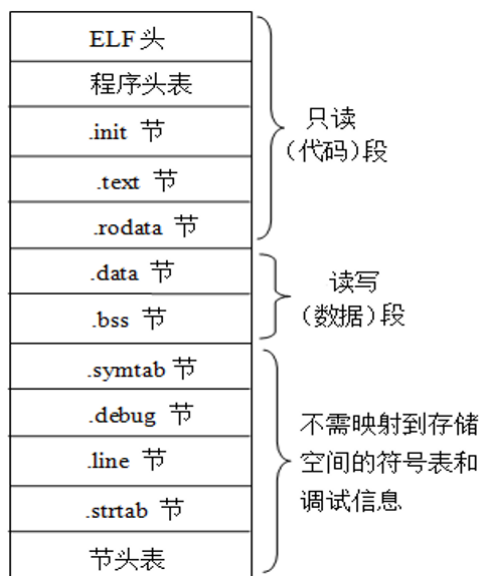
- 符号的定义与引用



- 链接过程的本质——合并相同的节



- 链接步骤
 - 符号解析 (Symbol resolution)
 - 编译器将定义的符号存放在一个符号表 (symbol table) 中
 - 链接器将每个符号的引用都与一个确定的符号定义建立关联
 - 重定位
 - 将多个代码段与数据段分别合并为一个单独的代码段和数据段
 - 计算每个定义的符号在虚拟地址空间中的绝对地址
 - 将可执行文件中符号引用处的地址修改为重定位后的地址信息
- 三类目标文件
 - 可重定位目标文件 (.o)
 - 其代码和数据可和其他可重定位文件合并为可执行文件
 - 每个.o 文件由对应的.c文件生成
 - 每个.o文件代码和数据地址都**从0开始**
 - 可执行目标文件 (默认为a.out)
 - 包含的代码和数据可以被**直接复制到内存**并被执行
 - 代码和数据地址为**虚拟地址空间**中的地址
 - 共享的目标文件 (.so)
 - 特殊的可重定位目标文件，能在装入或运行时被装入到内存并自动被链接，称为**共享库文件**
 - Windows 中称其为 Dynamic Link Libraries (**DLLs**)
- ELF文件
 - 简述：ELF (Executable and Linkable Format, 可执行和可链接格式) 是一种用于存储可执行文件、目标文件、共享库和核心转储的标准文件格式。它被广泛用于现代操作系统，如 Linux、FreeBSD、Solaris等。
 - 文件主要组成部分 (举例)



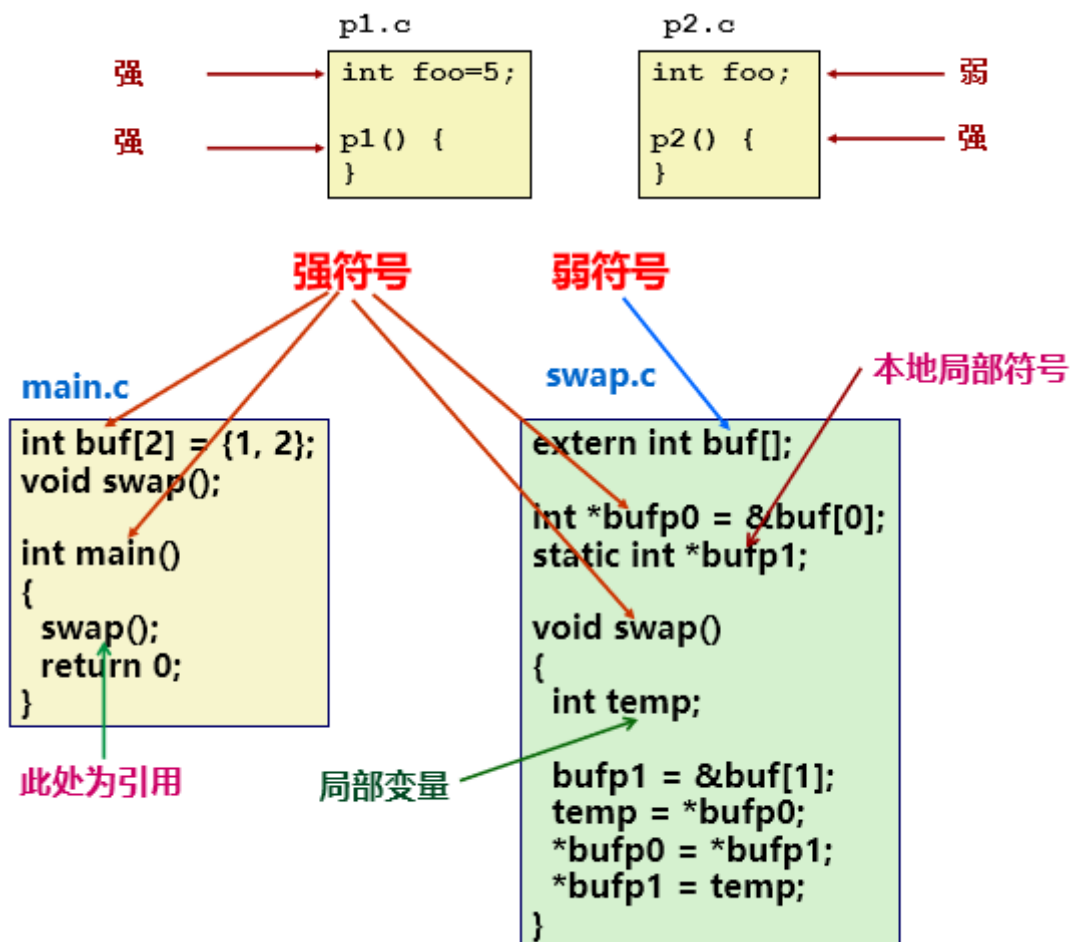
- ELF头：整个ELF文件的布局和类型，包括文件的入口点、程序头表和节头表的位置等
- 程序头表：文件的段（Segment）信息，用于加载器将文件加载到内存中。每个段包含一组相关的节（Section），用于运行时的内存映射。
- 节头表：描述了文件的节（Section）信息，用于链接器进行符号解析和链接操作。每个节包含一组相关的数据，如代码、数据、符号表等。
- 节：存储文件的各种数据，如代码、数据、符号表、重定位表等。
- 段：用于运行时的内存映射，每个段包含一组相关的节。
- 不同的类型：
 - 可执行文件：可以直接在操作系统上运行
 - 目标文件：用于链接器将多个目标文件链接成一个可执行文件或共享库
 - 共享库：可以在运行时被多个程序共享，减少内存占用和磁盘空间
 - 核心转储文件：记录程序运行时的内存状态，用于调试

4.2符号解析与重定位

4.2.1符号和符号表、符号解析

目的就是将符号的引用与符号的定义关联起来

- 链接器符号
 - 全局符号
 - 模块m定义的符号可以被其他模块引用
 - 如：非静态C函数和非静态全局变量
 - 外部符号
 - 由模块m引用但由其他模块定义的全局符号
 - 本地（局部）符号
 - 由模块m专门定义和引用的符号
 - 如：用静态属性定义的C函数和全局变量
- 符号强弱
 - 函数名和已初始化的全局变量名是强符号
 - 未初始化的全局变量名是弱符号



• 多重定义符号处理的三条规则

- 强符号不能多次定义，否则链接错误
- 若一个符号被定义为一次强符号和多次弱符号，则按强定义为准
- 若有多个弱符号定义，则任选其中一个

```
int x;
p1() {}
```

```
p1() {}
```

链接时错误：两个强符号 (p1)

```
int x;
p1() {}
```

```
int x;
p2() {}
```

对x的引用将引用相同的未初始化的int。这是程序员真正想要的吗？

```
int x;
int y;
p1() {}
```

```
double x;
p2() {}
```

在p2中写入x可能会覆盖y！

```
int x=7;
int y=5;
p1() {}
```

```
double x;
p2() {}
```

在p2中写入x将覆盖y！

```
int x=7;
p1() {}
```

```
int x;
p2() {}
```

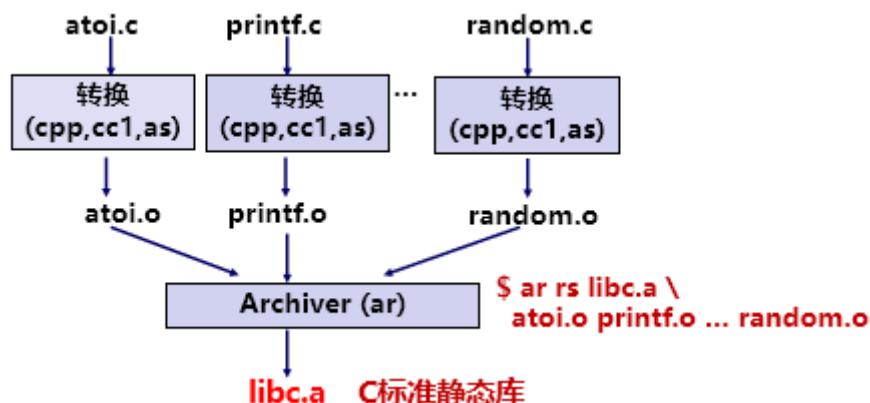
对x的引用将引用相同的初始化变量

4.2.2与静态库的链接

• 静态库文件

- 解释：将所有相关的目标模块 (.o) 打包为一个单独的库文件 (.a)，称为**静态库文件**，也称存档文件 (archive)
- **增强链接器**功能，使其能通过查找一个或多个库文件中的符号来解析符号

- 在构建可执行文件时只需**指定库文件名**，链接器会自动到库中寻找那些应用程序用到的目标模块，并且只把用到的模块从库中拷贝出来
- 在gcc命令行中**无需明显**指定C标准库libc.a(默认库)
- 静态库文件创建过程



- 链接器对外部引用的解析
 - 按照**命令行给出的顺序**扫描.o 和.a 文件
 - 扫描期间将当前未解析的引用记录到一个列表U中
 - 每遇到一个新的.o 或 .a 中的模块，都试图用其来解析U中的符号
 - 如果扫描到最后，U中还有未被解析的符号，则发生错误

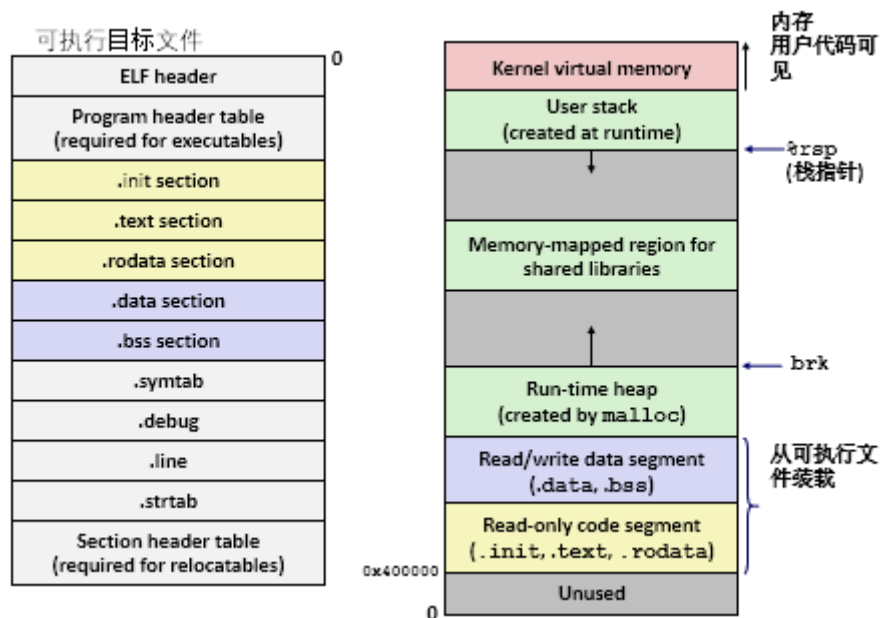
4.2.3重定位信息、重定位过程

目的是分别合并代码和数据，并根据代码和数据在虚拟地址空间中的位置，确定每个符号的最终存储地址，然后根据符号的确切地址来修改符号的引用处的地址

- 重定位步骤
 - 合并相同的节
 - 将集合E的所有目标模块中相同的节合并成新节
 - 例如，所有.text节合并作为可执行文件中的.text节
 - 对**定义符号**进行重定位（确定“符号定义”地址）
 - 确定新节中所有定义符号在虚拟地址空间中的地址
 - 例如，确定函数首地址，进而确定每条指令的地址及其变量首地址。
 - 完成这一步后，每条指令和每个全局变量都有确定的地址。
 - 对**引用符号**进行重定位（确定“符号引用”地址）
 - 修改.text节和.data节中对每个符号的引用（地址）
 - 需要用到在.rel_data和.rel_text节中保存的重定位信息

4.2.4可执行文件的加载

- 通过调用execve系统调用函数来调用加载器。
- 加载器（loader）根据可执行文件的程序（段）头表中的信息，将可执行文件的代码和数据从磁盘“拷贝”到存储器中。
- 加载后，将PC（EIP）设定指向Entry point（即符号_start处），最终执行main函数，以启动程序执行



4.3动态链接

- 动态链接进行方式
 - 在第一次加载并运行时进行 (load-time linking)
 - Linux通常由动态链接器(ld-linux.so)自动处理
 - 标准C库 (libc.so) 通常按这种方式动态被链接
 - 在已经开始运行后进行(run-time linking)
 - 在Linux中, 通过调用 dlopen()接口来实现
- 共享库优势
 - 节省内存空间: 在内存中只有一个备份, 被所有进程共享
 - 节省磁盘空间: 一个共享库目标文件被所有程序共享链接
 - 使用方便: 共享库升级时, 被自动加载到内存和程序动态链接
 - 效率高: 共享库可分模块、独立、用不同编程语言进行开发
 - 容易拓展: 第三方开发的共享库可作为程序插件